Guia para despliegues de APIs / Microservicios en AWS

Versión	Creador por	Fecha
1.0	Andrés Oswaldo Henríquez Gómez aohenriq@bancoagricola.com.sv	27/07/2026

Tabla de contenido

Guia para despliegues de APIs / Microservicios en AWS 1

Configuración del proyecto 2

- Dependencias necesarias 3
- Configuración del application.yaml 3
- Dockerfile 4
- Manifiestos de k8s 4
  - Deployment 5
  - Service 12
  - Horizontal Pod Autoescaler 14
  - Pod Disruption Budget 16
  - Service Account 17
  - Gateway 18
  - VirtualService 19
  - Vars 21
- Azure-pipelines 23
- Controladores 23

Configuración de IAC 24

- Recursos y métodos de APIGw 25
  - Recursos 25
  - Métodos 25
- Rol IRSA 26

Configuración de CI/CD 29

- Modificaciones al CI en Azure 30
- Modificaciones al CD 30
  - Ambiente DEV 30
  - Ambiente QA 40

Configuración del proyecto

Dependencias necesarias

En caso de utilizar elementos de AWS externos al propio EKS (Tablas de Dynamo, SQS, Secret Manager), deberá ser necesario crear un rol IRSA (IAM Roles for Service Accounts), que permite asignar permisos específicos de AWS a aplicaciones en pods de Kubernetes.

Para que dicho rol (Ver Rol IRSA) sea reconocido en los diferentes ambientes, debe de incluirse la librería de sts de AWS en el archivo de *build.gradle*:

```
dependencies {
       implementation 'software.amazon.awssdk:sts:2.25.0'
implementation 'software.amazon.awssdk:dynamodb-enhanced:2.25.0'
implementation 'software.amazon.awssdk:dynamodb:2.25.0'
}
```

De la misma manera, para realizar los respectivos healthchecks en el EKS, es necesaria la librería de springboot-starter-actuator:

```
implementation('org.springframework.boot:spring-boot-starter-actuator') {
exclude group: 'org.springframework.boot', module: 'spring-boot-starter-logging'
}
```

Configuración del application.yaml

Se deben configurar las opciones de puerto en que la aplicación se ejecutará de manera explícita para no chocar con otra aplicación ya implementada. Ver Inventario de APIs para determinar los puertos disponibles.

Y también se deben configurar permitir los healthcheks que realizará el EKS para determinar el estado del pod.

```
server:
  port:                                                                    8081

management:
  endpoint:
    health:
      probes:
        enabled: true
  endpoints:
    web:
      exposure:
        include: health
  health:
    livenessstate:
```

```
      enabled: true
    readinessstate:
      enabled: true
```

Dockerfile

Es un documento de texto plano que sirve como una receta con instrucciones paso a paso para crear una imagen de Docker. Sus funciones principales incluyen definir la imagen base, instalar programas y copiar archivos.

Para las APIS se debe utilizar la imagen de java proporcionada por el repositorio del banco. A fecha de la creación de este manual, la versión de java autorizada es **Java 21**, por lo que la imagen a importar es

dockeragentsbancoagricola.azurecr.io/baes/openjdk:21

Es importante mencionar que, al hacer uso de esta imagen el usuario que ejecuta esta imagen es el 8877 (Este dato nos sirve para la declaración de está configuración en el manifiesto de despliegue de k8s)

Dentro de este archivo se declara el directorio de trabajo de la aplicación y el puerto que se expondrá, que debe ser el mismo que se declaró en el archivo *application.yaml*.

El archivo .jar que se expondrá lo encontramos en la carpeta */miproyecto/build/libs/miProyecto.jar*

Ejemplo:

```
FROM dockeragentsbancoagricola.azurecr.io/baes/openjdk:21

WORKDIR /app

COPY AW1103046_PushInfoMgmt_CNA_GRD-1.0.0.jar ./application.jar

EXPOSE 8081

CMD [ "java", "-Duser.timezone=America/El_Salvador", "-Djava.io.tmpdir=/app/data", "-jar",
"application.jar" ]
```

Manifiestos de k8s

Para realizar el despliegue del API se requieren manifiestos k8s, estos son archivos de configuración en formato YAML que describen el estado deseado de los recursos y aplicaciones dentro de un clúster. Funcionan como planos o guías que le indican al sistema qué contenedores ejecutar, cuántas réplicas mantener y cómo deben comunicarse.

Para las apis son necesarios los siguientes manifiestos:

- Deployment
- Service
- Horizontal Pod Autoescaler
- Pod Disruption Budget
- Service Account (Si se conecta a un servicio de AWS fuera del EKS)
- Gateway
- VirtualService
- Vars (para manejar diferentes datos de manera dinámica según el ambiente)

Deployment

Es un archivo de configuración en formato YAML que define el estado deseado de una aplicación. Este archivo controla cuántas copias o réplicas de los contenedores deben ejecutarse y qué imagen usa.

Para el caso de nuestras aplicaciones es importante tener en cuenta los siguientes parámetros:

- name
- namespace
- labels
    - App
- spec
    - ServiceAccountName (En caso de utilizar un IRSA)
    - SecurityContext
        - RunAsUser
    - Containers
        - Image
        - Ports
            - ContainerPort
        - VolumeMounts
            - Name
            - MountPath
        - StartupProbe
        - LivenessProbe
        - ReadinessProbe
        - env

Name es el identificador único del objeto Deployment dentro de su namespace, permitiendo identificarlo y gestionarlo en el clúster. Es importante definir un nombre

unico por aplicación y persistirlo en las demás configuraciones donde se haga referencia a este.

Namespace define el espacio de nombres lógico o entorno virtual donde se ubicará el recurso dentro del clúster. Para nuestro caso, los namespaces en los diferentes ambientes son:

- devmtls-crosschannel, para DEV
- qamtls-crosschannel, para QA
- pendiente, para PROD

Labels: app es una etiqueta de metadatos clave-valor que sirve para identificar, agrupar y conectar recursos del clúster. Permite que objetos externos como los servicios (Service) encuentren los Pods correctos y facilita la gestión visual o por comandos. Al igual que name, es importante persistirlo en otros manifiestos, ya que con este valor es como se identifica el pod de nuestra aplicación.

Spec: ServiceAccountName define la identidad digital que usarán los contenedores para autenticarse y los permisos que tendrán al comunicarse con la API de Kubernetes o recursos externos. En este parametro asignamos el nombre de nuestro manifiesto ServiceAccount, que se describe más adelante. Es necesario declararlo si utilizamos un rol IRSA.

Spec: SecurityContext: runAsUser define el ID de usuario (UID) numérico con el que se ejecutarán los procesos dentro de los contenedores. Su función principal es evitar el uso del usuario root (UID 0) por defecto, mejorando la seguridad del clúster. Este deberá tener el valor de 8877 que se mencionaba en la sección del Dockerfile.

Spec: Containers: Image define la imagen de contenedor exacta que se descargará y ejecutará en los pods. Para nuestros despliegues se utiliza el ECR montado en AWS, seguido del nombre de la imagen que se guardó (Este nombre se configura en el CD. (Ver Modificaciones al CD) en formato *Repositorio:NombreImagen*

Los repositorios según ambiente de CrossChannel son:

- 730335499576.dkr.ecr.us-east-1.amazonaws.com/ecr-dev-mysismstraza-mcjzkr4ltij3, para DEV
- 180250392861.dkr.ecr.us-east-1.amazonaws.com/ecr-qa-mysismstraza-1sq0ofpjajvx, para QA
- Pendiente el de producción

Spec: Containers: Ports: ContainerPort indica el número de puerto en el que la aplicación escucha activamente dentro del contenedor. Debe tener el mismo puerto que se declaró en el *application.yaml* y el *Dockerfile*.

Spec: Containers: VolumeMounts especifica en qué ruta exacta del sistema de archivos interno de un contenedor se conectará un volumen definido a nivel de Pod. Permite que el contenedor lea y escriba datos en dicho almacenamiento. Declara dos parámetros:

- **name**: El nombre del volumen declarado en la sección de volúmenes del Pod.
- **mountPath**: La ruta absoluta dentro del contenedor donde aparecerá el contenido del volumen.

Para nuestras aplicaciones es importante definir 3 volúmenes, donde se aloja la aplicación, donde se alojan los logs y donde se aloja el secreto de acceso del rol IRSA si creamos uno.

- Alojamiento de aplicación
    - Name: data-volumen
    - MountPath: /app/data (Si observamos, es el mismo directorio declarado en el Dockerfile)
- Alojamiento de logs
    - Name: logs-volume
    - MountPath: /app/logs
- Alojamiento de secreto de rol IRSA
    - Name: aws-token
    - MountPath: /var/run/mi_api_irsa

Spec: Containers: StartupProbe es una sonda de inicio que verifica si la aplicación dentro de un contenedor ya terminó de arrancar. Mientras startupProbe no dé un resultado exitoso, las sondas de vida (liveness) y disponibilidad (readiness) permanecen desactivadas, evitando que Kubernetes mate por error a contenedores lentos.

Se ejecuta exclusivamente al iniciar el contenedor. Si la aplicación tarda en cargar datos pesados o dependencias, le da el tiempo necesario sin interferir.

Si supera el límite de intentos fallidos permitidos, Kubernetes mata el contenedor y aplica su política de reinicio. Una vez que pasa con éxito, cede el control por completo a las otras sondas.

Dentro de su configuración se declara el path */actuator/health,* que es el que se definió en nuestro *application.yaml*.

Spec: Containers: LivenessProbe es una sonda de actividad que verifica periódicamente si la aplicación dentro de un contenedor está funcionando correctamente o si se ha quedado bloqueada. Si la prueba falla, el componente kubelet reinicia automáticamente el contenedor.

Detecta estados de bloqueo interno (deadlocks) o fallos irrecuperables donde el proceso del sistema operativo sigue activo pero la aplicación ya no responde a los usuarios.

Si se acumulan los fallos configurados, Kubernetes destruye el contenedor actual y crea uno nuevo siguiendo la política de reinicio.

Dentro de su configuración se declara el path */actuator/health/liveness,* que es el que se definió en nuestro *application.yaml*.

Spec: Containers: ReadinessProbe es una sonda de control que le indica a Kubernetes si un contenedor está listo para recibir tráfico de red, retirándolo de los servicios si la prueba falla. Evita enviar peticiones a un pod que aún está cargando datos o conectado a bases de datos.

Remueve el pod de los Endpoints del Service sin reiniciar el contenedor y vuelve a incluir el pod en el balanceo cuando la aplicación responde de forma exitosa.

Dentro de su configuración se declara el path */actuator/health/ readiness,* que es el que se definió en nuestro *application.yaml*.

Spec: Containers: env define variables de entorno que se inyectan directamente dentro del contenedor en tiempo de ejecución. Permite configurar la aplicación sin modificar su código fuente.

Para nuestros despliegues es importante declarar las variables:

- **TZ**: America/El_Salvador
- **AWS_ROLE_ARN**: ARN del rol IRSA que se haya creado
- **AWS_WEB_IDENTITY_TOKEN_FILE**: /var/run/mi_api_irsa/token
- **AWS_REGION**: us-east-1

Ejemplo:

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: $(appName)-deploy
  namespace: $(namespace)
  labels:
    app: $(appName)

spec:
```

```
  replicas: $(replicas) #Cantidad de pods que permanecerán activos
simultáneamente.
  minReadySeconds: $(minReadySeconds) #Tiempo mínimo que un pod debe permanecer
en estado Ready antes de considerarse disponible.

  #Estrategia de actualizacion
  strategy:
    type: RollingUpdate #Permite actualizar los pods sin interrupción del
servicio.
    rollingUpdate:
      maxUnavailable: 0 #No permite que existan pods indisponibles durante la
actualización.
      maxSurge: 25% #Permite crear hasta un 25% de pods adicionales durante el
proceso de despliegue.
  selector:
    matchLabels:
      app: $(appName) #Relaciona el Deployment con los pods que poseen la
etiqueta app

  template:
    metadata:
      labels:
        app: $(appName)

    spec:
      serviceAccountName: $(appName)-service-account
      automountServiceAccountToken: false #Evita que Kubernetes monte
automáticamente el token del Service Account por motivos de seguridad.
      terminationGracePeriodSeconds: 60 #Otorga hasta 60 segundos para finalizar
procesos antes de eliminar el pod.

      securityContext:
        runAsNonRoot: true #Otorga hasta 60 segundos para finalizar procesos
antes de eliminar el pod.
        runAsUser: $(run_as_usser) #UID del usuario que ejecutará el proceso.
Este usuario es el que la imagen JDK de baes monta

      topologySpreadConstraints: #Garantiza una distribución equilibrada de los
pods.
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone #Distribuye pods entre
diferentes zonas de disponibilidad.
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: $(appName)
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname #Distribuye pods entre distintos
nodos.
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: $(appName)

      containers:
        - name: $(appName)
          image: $(repository):PushInfoMgmt-latest
          imagePullPolicy: Always #Fuerza la descarga de la imagen en cada
despliegue.
```

```
          ports:
            - containerPort: $(containerPort) #Puerto expuesto por el
contenedor.

          resources:
            requests: #Recursos mínimos garantizados para el contenedor.
              cpu: $(requests.cpu)
              memory: $(requests.memory)
            limits: #Recursos máximos permitidos para el contenedor.
              cpu: $(limits.cpu)
              memory: $(limits.memory)

          volumeMounts:
            - name: aws-token #Almacena el token utilizado por IRSA (IAM Roles
for Service Accounts).
              mountPath: /var/run/push_info_mgmt_irsa
              readOnly: true
            - mountPath: /tmp #Directorio temporal en memoria.
              name: tmp
              subPath: tmp
            - mountPath: /app/data #Directorio para almacenamiento temporal de
datos de la aplicación.
              name: data-volume
            - mountPath: /app/logs #Directorio para almacenamiento temporal de
logs de la aplicación.
              name: logs-volume

          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            runAsNonRoot: true
            runAsUser: $(run_as_usser)
            capabilities:
              drop:
                - ALL #Elimina todas las capacidades privilegiadas del sistema
operativo para minimizar riesgos de seguridad.

          startupProbe: #Verifica que la aplicación haya iniciado correctamente.
            httpGet:
              path: /actuator/health
              port: $(containerPort)
            failureThreshold: 30
            periodSeconds: 10

          lifecycle:
            preStop:
              exec:
                command: [ "sh", "-c", "sleep 5" ] #Antes de finalizar el pod,
se espera 5 segundos para permitir que el balanceador deje de enviar solicitudes
activas.

          livenessProbe: #Determina si la aplicación sigue funcionando.
            httpGet:
              path: /actuator/health/liveness
              port: $(containerPort)
            initialDelaySeconds: 60
            periodSeconds: 15
            timeoutSeconds: 5
```

```
            failureThreshold: 3 #Si falla tres veces consecutivas, Kubernetes
reiniciará el contenedor.

          readinessProbe: #Determina si la aplicación está lista para recibir
tráfico.
            httpGet:
              path: /actuator/health/readiness
              port: $(containerPort)
            initialDelaySeconds: 30
            periodSeconds: 10
            timeoutSeconds: 5
            failureThreshold: 3

          env:
            - name: TZ
              value: America/El_Salvador
            - name: AWS_ROLE_ARN
              value: $(arn_rol_irsa)
            - name: AWS_WEB_IDENTITY_TOKEN_FILE
              value: /var/run/push_info_mgmt_irsa/token
            - name: AWS_REGION
              value: us-east-1

      volumes:
        - name: aws-token
          projected:
            sources:
              - serviceAccountToken: #Genera dinámicamente un token compatible
con AWS STS para el mecanismo IRSA.
                  path: token
                  audience: sts.amazonaws.com
                  expirationSeconds: 86400
        - name: tmp
          emptyDir:
            medium: Memory #Volumen temporal almacenado completamente en memoria
RAM.
            sizeLimit: 64Mi
        - name: data-volume
          emptyDir: {} #Volumen efímero utilizado para almacenamiento temporal
de información mientras el pod se encuentra activo.
        - name: logs-volume
          emptyDir: {} #Volumen efímero utilizado para almacenamiento temporal
de logs mientras el pod se encuentra activo.
```

Service

Es un archivo de configuración en formato YAML que define cómo conectar y dar acceso estable a un grupo de Pods. Como los Pods nacen, mueren y cambian de dirección IP constantemente, el Service actúa como una puerta de enlace fija, uniendo una IP estable y un nombre de red interno con esos Pods cambiantes mediante un balanceador de carga.

Para el caso de nuestras aplicaciones es importante tener en cuenta los siguientes parámetros:

- name
- namespace
- labels
    - App
- spec
    - Type
    - Ports
        - Port
        - TargetPort

Name es el identificador único del objeto Service dentro de su namespace, permitiendo identificarlo y gestionarlo en el clúster. Es importante definir un nombre único por aplicación y persistirlo en las demás configuraciones donde se haga referencia a este.

Namespace define el espacio de nombres lógico o entorno virtual donde se ubicará el recurso dentro del clúster.

Labels: app es una etiqueta de metadatos clave-valor que sirve para identificar, agrupar y conectar recursos del clúster. Al igual que name, es importante persistirlo en otros manifiestos, ya que con este valor es como se identifica el service de nuestra aplicación.

Spec: Type define cómo se expone el servicio a la red, ya sea de forma interna dentro del clúster o hacia el exterior. Los valores principales son ClusterIP, NodePort y LoadBalancer.

Para nuestras aplicaciones utilizaremos el valor **ClusterIP,** este asigna una IP interna y virtual del clúster. Solo permite la comunicación entre recursos o Pods dentro del mismo clúster y no es accesible desde fuera. Posteriormente con El manifiesto de VirtualService y Gateway configuraremos la comunicación del pod hacia el exterior.

Spec: Ports: Port define el puerto en el que el propio servicio va a escuchar las solicitudes dentro del clúster. Funciona como el punto de acceso virtual al que otros pods se conectan

Es el número de puerto que se asigna a la IP virtual (ClusterIP) del servicio. Port es el puerto público del servicio, mientras que targetPort es el puerto real en el que corre la aplicación dentro del contenedor del Pod hacia donde se reenvía el tráfico.

Para efectos prácticos, en nuestras aplicaciones utilizamos el mismo puerto en ambos parámetros, y este puerto es el declarado en *application.yaml* y *Dockerfile*.

Spec: Ports: TargetPort es el puerto en el que la aplicación escucha de verdad dentro del Pod o contenedor. Define hacia dónde se debe reenviar el tráfico que recibe el servicio.

Para efectos prácticos, en nuestras aplicaciones utilizamos el mismo puerto en ambos parámetros, y este puerto es el declarado en *application.yaml* y *Dockerfile*.

Ejemplo:

```
apiVersion: v1
kind: Service
metadata:
  name: $(appName)-service
  namespace: $(namespace)
  labels:
    app: $(appName)

spec:
  type: ClusterIP
  internalTrafficPolicy: Cluster
  ipFamilies:
    - IPv4
  ipFamilyPolicy: SingleStack
  selector:
    app: $(appName)
  sessionAffinity: None
  ports:
    - name: service-port-internal
      protocol: TCP
      port: $(containerPort)
      targetPort: $(containerPort)
```

Horizontal Pod Autoescaler

Es un archivo de configuración en formato YAML que define las indicaciones para que el sistema que aumente o disminuya de manera automática la cantidad de réplicas (pods) de una aplicación basándose en el uso real de recursos como la CPU o la memoria.

Para el caso de nuestras aplicaciones es importante tener en cuenta los siguientes parámetros:

- name
- namespace
- spec
    - ScalableTargetRef
        - kind
        - name

Name es el identificador único del objeto HPA dentro de su namespace, permitiendo identificarlo y gestionarlo en el clúster. Es importante definir un nombre único por aplicación y persistirlo en las demás configuraciones donde se haga referencia a este.

Namespace define el espacio de nombres lógico o entorno virtual donde se ubicará el recurso dentro del clúster.

Spec: ScalableTargetRef: Kind define el tipo de recurso al que se le aplicará el autoescalado. En nuestro caso se usa *Deployment*.

Spec: ScalableTargetRef: Name define el nombre exacto del recurso objetivo al que el HPA va a monitorear y ajustar el número de réplicas. En nuestro caso, debe ser el mismo nombre que definimos en el manifiesto de Deployment.

Ejemplo:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: $(name)-hpa
  namespace: $(namespace)
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: $(appName)-deploy
  minReplicas: $(min) #Numero minimo de replicas permitidas
  maxReplicas: $(replicas) #Numero maximo de replicas permitidas para el
escalamiento
  metrics:
    - type: Resource
      resource:
        name: cpu #Monitorea la utilizacion promedio de CPU de los pods
        target:
          type: Utilization
```

```
          averageUtilization: 50 #Si el consumo promedio supera el 50 % de la
solicitud de CPU configurada en los contenedores, Kubernetes incrementara el
número de replicas
    - type: Resource
      resource:
        name: memory #Monitorea la utilizacion promedio de memoria de los pods
        target:
          type: Utilization #Si el consumo promedio supera el 50 % de la memoria
solicitada, el HPA incrementara el número de replicas
          averageUtilization: 50
```

Pod Disruption Budget

Es un archivo de configuración en formato YAML que limita la cantidad de pods de una aplicación que pueden estar inactivos al mismo tiempo. Protege la disponibilidad del servicio durante interrupciones voluntarias, como el mantenimiento de nodos o actualizaciones del clúster. El PDB evita que demasiados pods de una aplicación sean interrumpidos simultáneamente, asegurando la continuidad del servicio.

Para el caso de nuestras aplicaciones es importante tener en cuenta los siguientes parámetros:

- name
- namespace
- labels
    - App

Name es el identificador único del objeto PDB dentro de su namespace, permitiendo identificarlo y gestionarlo en el clúster. Es importante definir un nombre único por aplicación y persistirlo en las demás configuraciones donde se haga referencia a este.

Namespace define el espacio de nombres lógico o entorno virtual donde se ubicará el recurso dentro del clúster.

Labels: app es una etiqueta de metadatos clave-valor que sirve para identificar, agrupar y conectar recursos del clúster. Al igual que name, es importante persistirlo en otros manifiestos, ya que con este valor es como se identifica el service de nuestra aplicación.

Ejemplo:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: $(appName)-pdb
  namespace: $(namespace)
  labels:
    app: $(appName)
spec:
  minAvailable: $(pdbMin) #Numero minimo de pods que deben estar en ejecucion y
disponibles
  selector:
    matchLabels:
      app: $(appName) #Aplica la configuración a los pods que posean las
etiquetas especificadas
```

Service Account

Es un archivo de configuración en formato YAML que define una identidad digital a los procesos que corren dentro de un Pod, permitiéndoles comunicarse de forma segura con la API del clúster. Sirve para dar identidad a los pods, controlar permisos y separar accesos por aplicación.

Para el caso de nuestras aplicaciones es importante tener en cuenta los siguientes parámetros:

- name
- namespace
- annotations
    - eks.amazonaws.com/role-arn

Name es el identificador único del objeto PDB dentro de su namespace, permitiendo identificarlo y gestionarlo en el clúster. Es importante definir un nombre único por aplicación y persistirlo en las demás configuraciones donde se haga referencia a este.

Namespace define el espacio de nombres lógico o entorno virtual donde se ubicará el recurso dentro del clúster.

Annotations: eks.amazonaws.com/role-arn vincula la cuenta de servicio de un clúster de Amazon EKS a un rol específico de AWS IAM. Permite que los Pods que usen esa cuenta asuman de forma segura los permisos de AWS definidos en el rol.

Para nuestras aplicaciones, incluiremos el ARN del rol IRSA creado.

Ejemplo:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: $(appName)-service-account
  namespace: $(namespace)
  annotations:
    eks.amazonaws.com/role-arn: $(arn_rol_irsa)
```

Gateway

Es un archivo de configuración en formato YAML que define un punto de entrada de red (como un balanceador de carga) que gestiona y dirige el tráfico que viene desde fuera del clúster hacia los servicios internos.

Para el caso de nuestras aplicaciones es importante tener en cuenta los siguientes parámetros:

- name
- namespace
- spec
    - Selector
        - Istios

Name es el identificador único del objeto Gateway dentro de su namespace, permitiendo identificarlo y gestionarlo en el clúster. Es importante definir un nombre único por aplicación y persistirlo en las demás configuraciones donde se haga referencia a este.

Namespace define el espacio de nombres lógico o entorno virtual donde se ubicará el recurso dentro del clúster.

Spec: Selector: Istios es una etiqueta que identifica el Ingress Gateway que procesara el trafico entrante. Indica a Istio a qué pods del proxy Envoy (como el ingress gateway) debe enviar esta configuración de red.

Para nuestras aplicaciones el valor de esta etiqueta debe ser **ingressgateway**.

Ejemplo:

```
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: $(appName)-internal-gateway
  namespace: $(namespace)
spec:
  selector:
    istio: $(ingress_gateway_name) #Etiqueta que identifica el Ingress Gateway
que procesara el trafico entrante.
  servers:
  - port: #Acepta trafico HTTP por el puerto 80 desde cualquier host
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - "*"
```

VirtualService

Es un archivo de configuración en formato YAML que no es propio de Kubernets sino un recurso personalizado (CRD) introducido por herramientas de malla de servicios como Istio. Permite definir reglas avanzadas de enrutamiento de tráfico, dividir porcentajes de carga entre versiones y redirigir peticiones basadas en cabeceras HTTP.

Para el caso de nuestras aplicaciones es importante tener en cuenta los siguientes parámetros:

- name
- namespace
- spec
    - Gateways
    - Http
        - Match
            - Uri
                - Prefix
        - Route
            - Destination
                - Host
                - Port
                    - Number

Name es el identificador único del objeto Gateway dentro de su namespace, permitiendo identificarlo y gestionarlo en el clúster. Es importante definir un nombre único por aplicación y persistirlo en las demás configuraciones donde se haga referencia a este.

Namespace define el espacio de nombres lógico o entorno virtual donde se ubicará el recurso dentro del clúster.

Spec: Gateways define a qué puertas de enlace (gateways) o mallas (mesh) se aplican las reglas de enrutamiento. Sirve para conectar el tráfico externo o interno con los destinos del clúster.

Para nuestras aplicaciones debemos poner el nombre que asignamos al manifiesto *Gateway*.

Spec: Http: Match: Uri: Prefix sirve para filtrar y seleccionar peticiones HTTP basándose en si la ruta (URI) de la solicitud comienza con un texto o prefijo específico. Por ejemplo, si se configura el prefix: "/api/v1", cualquier llamada como /api/v1/users o /api/v1/items/12 cumplirá la condición.

Las reglas se leen de arriba a abajo; la primera coincidencia que encuentra el proxy se aplica.

Para nuestras aplicaciones debemos tomar en consideración que la ruta que pongamos como prefijo en este manifiesto, también debe configurarse en los controladores del API (Ver Controladores) y no debería usarse un prefijo ya utilizado (Ver Inventario de APIs para revisar los prefijos ya utilizados).

Spec: Http: Route: Destination: Host define el nombre del servicio de destino real hacia donde se enviará el tráfico HTTP o gRPC una vez que cumple con las reglas de enrutamiento.

Para nuestras aplicaciones este nombre siemore tiene la forma *mi_aplicacion-service.namespace.svc.cluster.local*. Donde todos comparten el namespace del ambiente y cada uno debe tener su propio nombre de aplicación para que el identificador sea único.

Spec: Http: Route: Destination: Host: Port: Number define el puerto numérico exacto del servicio de Kubernetes de destino al que se debe enviar la solicitud HTTP en un VirtualService de Istio. Nuevamente en nuestras aplicaciones debe ser el puerto definido en *application.yaml* y *Dockerfile*.

Ejemplo:

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: $(appName)-vs-internal
  namespace: $(namespace)
spec:
  hosts:
    - "*"
  gateways:
    - $(appName)-internal-gateway
  http:
    - match:
        - uri:
            prefix: /push-notification-management

      route:
        - destination:
            host: $(appName)-service.$(namespace).svc.cluster.local
            port:
              number: $(containerPort)
```

Vars

Es un archivo de configuración en formato YAML que define las variables a utilizar en los manifiestos k8s.

Para nuestras aplicaciones, siguiendo los pasos para configurar el CD (Ver Modificaciones al CD) debemos tener dos archivos de variables, uno para los manifiestos que se despliegan con el ServiceConecction de Operación y otro para el ServiceConnection de Fullstack.

Por ende, debemos tener una estructura de carpetas similar a

k8s_operacion: deployment.yaml, service.yaml, hpa.yaml, pdb.yaml, serviceAccount.yaml, vars.yaml

k8s_fullstack: gateway.yaml, virtualService.yaml, vars.yaml

Ambas carpetas deben ser exportadas mediante la configuracion del pipeline del proyecto en el archivo azure-pipelines (Ver Azure-pipelines)

Las variables sugeridas para el SVC de Operación son (Importante: tomar como base las variables, no sus valores):

```
dev:
  appName: mi-aplicacion
  namespace: namespace_desarrollo
  containerPort: 8081 #Puerto expuesto por la aplicacion
  arn_rol_irsa: arn:aws:iam::1234:role/nombre-del-rol-creado
  run_as_usser: 8877
```

```
  pdbMin: 1
  min: 1
  replicas: 1
  minReadySeconds: 7
  limits:
    memory: "2048Mi"
    cpu: "1000m"
  requests:
    memory: "1024Mi"
    cpu: "500m"
qa:
  appName: mi-aplicacion
  namespace: namespace_qa
  containerPort: 8081 #Puerto expuesto por la aplicacion
  arn_rol_irsa: arn:aws:iam::5678:role/nombre-del-rol-creado
  run_as_usser: 8877

  pdbMin: 1
  min: 2
  replicas: 4
  minReadySeconds: 7
  limits:
    cpu: "1500m"
    memory: "4Gi"
  requests:
    cpu: "1000m"
    memory: "2048Mi"
prod:
  appName: mi-aplicacion
  namespace: namespace_produccion
  containerPort: 8081 #Puerto expuesto por la aplicacion
  arn_rol_irsa: arn:aws:iam::9876:role/nombre-del-rol-creado
  run_as_usser: 8877

  pdbMin: 2
  min: 3
  replicas: 6
  minReadySeconds: 7
  limits:
    memory: "5120Mi"
    cpu: "1600m"
  requests:
    cpu: "1500m"
    memory: "4096Mi"
```

Las variables sugeridas para el SVC de Fullstack son (Importante: tomar como base las variables, no sus valores):

```
dev:
  appName: mi-aplicacion
  namespace: namespace_desarrollo
  containerPort: 8081 #Puerto expuesto por la aplicacion
  ingress_gateway_name: ingressgateway
qa:
  appName: mi-aplicacion
  namespace: namespace_qa
  containerPort: 8081 #Puerto expuesto por la aplicacion
  ingress_gateway_name: ingressgateway
prod:
  appName: mi-aplicacion
  namespace: namespace_produccion
  containerPort: 8081 #Puerto expuesto por la aplicacion
  ingress_gateway_name: ingressgateway
```

Azure-pipelines

Para el correcto funcionamiento de los Ci y CD del proyecto, se deben cambiar o agregar algunos valores al archivo de *azure-pipelines.yaml* incluidos en el directorio del proyecto. Estos son el appName, la opcion de publicar archivos y los archivos a exportar del lado del proyecto.
Marcado en negrita quedaría de la siguiente manera:

```
          - template: templates/gradlew.yml@templates
            parameters:
              appName: '$(System.DefaultWorkingDirectory)'
              publishFiles: true
              routeCopyFiles:
                - folder: 'build/libs'
                - source: 'k8s/*'
                - source: 'k8s_fullstack/*'
                - file: 'Dockerfile'
```

Controladores

Como se mencionaba en VirtualService, para el correcto funcionamiento del prefix también es necesario que nuestros controladores estén configurados para manejar este prefix.

Por ejemplo, si en la configuracion del VirtualService asignamos el prefix de */push-notification-manegement*, todos los controladores de nuestra aplicación deberán tener este prefix, de la siguiente manera.

```
@RequestMapping("/push-notification-management/push-management-channel")
public class PushNotificationChannelController{

@RequestMapping("/push-notification-management/push-management-customer")
public class PushNotificationClientController {
```

Por último, para el correcto funcionamiento de toda esta configuración, este paso se complementará en las configuraciones de los métodos del ApiGw.

Configuración de IAC

Recursos y métodos de APIGw

Recursos

Cada desarrollador será responsable de crear los recursos (rutas) y métodos (Endpoints) en el APiGw mediante IAC. Para eso será necesario crear un archivo único por aplicación que despliegue estos recursos.

Primero hay que tener en cuenta que en el proyecto de CrossChannel ya existe un archivo que crea recursos iniciales en el APIGw, estos recursos forman dos rutas:

- /int/v1/sales-and-service/cross-channel/outbound-messaging/
- /int/v1/sales-and-service/customer-management/

Es a partir de estas rutas donde nacerán las nuevas rutas para las APIS. Para hacer el enlace a estas rutas, por ejemplo, para crear un nuevo recurso a partir de *customer-management*, debemos asignar este recurso al nuevo en el parametro de ParentId:

```
ApiGatewayCustomerAccessEntitlement:
    Type: AWS::ApiGateway::Resource
    Properties:
      ParentId: !ImportValue "ApiGW-${environmentTag}-
customerManagementResource"
      PathPart: customer-access-entitlement
      RestApiId: !ImportValue "ApiGW-${environmentTag}-RestApiId"
```

Podemos ver la lista de nombres exportados en el archivo *apigw-core.yaml* del repositorio de IAC de CrossChannel. En caso de que para un API en específico la ruta deba empezar desde otro recurso anterior, podemos obtener su nombre exportado en este mismo archivo y aplicarlo de la misma manera que en el ejemplo.

Métodos

Para el caso de los métodos, la estructura es la siguiente:

```
ApiGatewayPushManagementCustomerPost:
    Type: AWS::ApiGateway::Method
    Properties:
      AuthorizationType: NONE
      ApiKeyRequired: false
      HttpMethod: POST
      Integration:
        Type: HTTP_PROXY
        IntegrationHttpMethod: POST
        ConnectionType: VPC_LINK
        ConnectionId: !ImportValue "ApiGW-${environmentTag}-VpcLinkId"
        Uri: !Sub "http://${nlb_dns_name}/push-notification-management/push-
management-customer"
```

```
      ResourceId: !Ref ApiGatewayPushManagementCustomer
      RestApiId: !ImportValue "ApiGW-${environmentTag}-RestApiId"
```

Los valores a tener en cuenta dentro del bloque de **Integration** son:

- **Type HTTP_PROXY**, significa que el servicio actúa como un "pasarela transparente". API Gateway toma la petición que recibe del cliente y la envía de forma directa a una URL backend HTTP externa o interna, sin modificar ni procesar el contenido de la solicitud o la respuesta.
- **ConnectionType VPC_LINK**, significa que el API utiliza una integración privada. Esto le permite a API Gateway comunicarse de forma segura con recursos alojados dentro de la red privada virtual (VPC) sin exponer dichos recursos a la internet pública.
- **ConnectionId**, en este parámetro colocamos el valor exportado de un VPCLink previamente creado en el proyecto de CrossChannel, estre VPCLink apunta al ARN del balanceador de carga de Istios, que el equipo de nube creo previamente.
- **Uri**, en este parámetro colocamos el prefijo *http://*, seguido del nombre de dominio del balanceador de carga de Istios, que el equipo de nube creo previamente. Por último, se concatena el prefix que configuramos en nuestros VirtualService y en nuestros controladores.
  Con estos 3 elementos configurados, aseguramos la segmentación de todas nuestras APIs, para que las peticiones entrantes mediante el APIGw se redirijan al API en específico que se desea consumir.
- **ResourceId**, es el nombre del recurso que precede al endpoint que estamos configurando.

Adicional, dentro del mismo archivo se crea un recurso de despliegue que apunte al stage previamente creado en el proyecto de CrossChannel. La creación de este recurso dependerá de que todos los recursos y métodos se creen con éxito previamente.

```
ApiGatewayDeploymentPushInfoMgmtV5:
    Type: AWS::ApiGateway::Deployment
    DependsOn:
      - ApiGatewayCustomerAccessEntitlement
      - ApiGatewayPushNotificationManagement
    Properties:
      Description: !Ref BuildId
      RestApiId: !ImportValue "ApiGW-${environmentTag}-RestApiId"
      StageName: !ImportValue "ApiGW-${environmentTag}-StageName"
```

Cada vez que se cree un cambio en los recursos y metodos de este archivo, es necesario cambiar el nombre del recurso de despliegue, esto para que la plataforma de AWS lo tome como un nuevo recurso y actualice el Stage, que es el que permite que se reciban solicitudes al APIGW.

Rol IRSA

En caso de necesitar comunicarse desde el API alojada en el EKS hacia otro componente como tablas de DynamoDB, secretos de SecretManager, etc. Es necesario crear un rol que permita la interacción desde el API en el EKS hacia el recurso correspondiente.

Un rol IRSA (IAM Roles for Service Accounts) es una función de seguridad de Amazon Web Services (AWS) que permite asociar un rol de IAM directamente a una cuenta de servicio de Kubernetes. Esto permite que los contenedores o pods de un clúster de Amazon EKS accedan a servicios de AWS con permisos específicos sin usar credenciales.

Como ejemplo, un rol que permite la comunicación a diferentes tablas de base de datos:

```
AWSTemplateFormatVersion: '2010-09-09'
Description: 'IRSA setup para interactuar con la base de datos desde el EKS para
el proyecto de PushInfoMgmt'

Parameters:
  Environment:
    Description: Parametro de ambiente (dev/qa/prod)
    Type: String
    AllowedValues: [dev, qa, pdn]
    Default: dev

Conditions:
  IsDev: !Equals [!Ref Environment, dev]

Resources:
  EksDynamoCrossIrsaRolePushMgmt:
    Type: AWS::IAM::Role
    Properties:
      PermissionsBoundary: !Sub arn:aws:iam::${AWS::AccountId}:policy/Lz-
Governance-Boundary
      RoleName: !Sub EKS-001-CROSSCHANNEL-${environmentTag}-push-info-mgmt-
service-account-irsa
      AssumeRolePolicyDocument:
        Version: '2012-10-17'
        Statement:
          - Effect: Allow
            Principal:
              Federated: "${federated_oidc_provider_arn}"
            Action: sts:AssumeRoleWithWebIdentity
            Condition:
              StringEquals:
                "${oidc_provider_url}:aud": "sts.amazonaws.com"
                "${oidc_provider_url}:sub": !Sub
"system:serviceaccount:${namespace}:push-info-mgmt-service-account"
      Policies:
      - PolicyName: kmsAccess
        PolicyDocument:
          Version: '2012-10-17'
          Statement:
            - Effect: Allow
              Action:
```

```
                - kms:Decrypt
                - kms:DescribeKey
              Resource:
                !If
                - IsDev
                - !ImportValue "${applicationcode}-kms-${environment}-arn"
                - !ImportValue "${applicationcode}-dynamo-${environment}-arn"

      - PolicyName: dynamodbAccess
        PolicyDocument:
          Version: '2012-10-17'
          Statement:
            - Effect: Allow
              Action:
              - dynamodb:GetItem
              - dynamodb:GetRecords
              - dynamodb:PutItem
              - dynamodb:Scan
              - dynamodb:Query
              - dynamodb:UpdateItem
              Resource:
              - !ImportValue "${applicationcode}-ddb-001-pushmgmt-
${environment}-arn"
              - !Join
                  - ""
                  - - !ImportValue
                        Fn::Sub: "${applicationcode}-ddb-001-pushmgmt-
${environment}-arn"
                    - "/index/*"

              - !ImportValue "${applicationcode}-ddb-002-pushmgmt-
${environment}-arn"
              - !Join
                  - ""
                  - - !ImportValue
                        Fn::Sub: "${applicationcode}-ddb-002-pushmgmt-
${environment}-arn"
                    - "/index/*"

              - !ImportValue "${applicationcode}-ddb-003-pushmgmt-
${environment}-arn"
              - !Join
                  - ""
                  - - !ImportValue
                        Fn::Sub: "${applicationcode}-ddb-003-pushmgmt-
${environment}-arn"
                    - "/index/*"

              - !ImportValue "${applicationcode}-ddb-004-pushmgmt-
${environment}-arn"
              - !Join
                  - ""
                  - - !ImportValue
                        Fn::Sub: "${applicationcode}-ddb-004-pushmgmt-
${environment}-arn"
                    - "/index/*"


      Tags:
```

```
      - Key: baes:requested-by
        Value: ${baesRequestedBy}
      - Key: baes:di-o-de
        Value: ${baesDiODe}
      - Key: baes:provided-by
        Value: 'Andres Oswaldo Henriquez Gomez'
      - Key: baes:creation-date
        Value: '23-06-2026'

Outputs:
  RolName:
    Description: Rol ARN
    Value: !GetAtt EksDynamoCrossIrsaRolePushMgmt.Arn
```

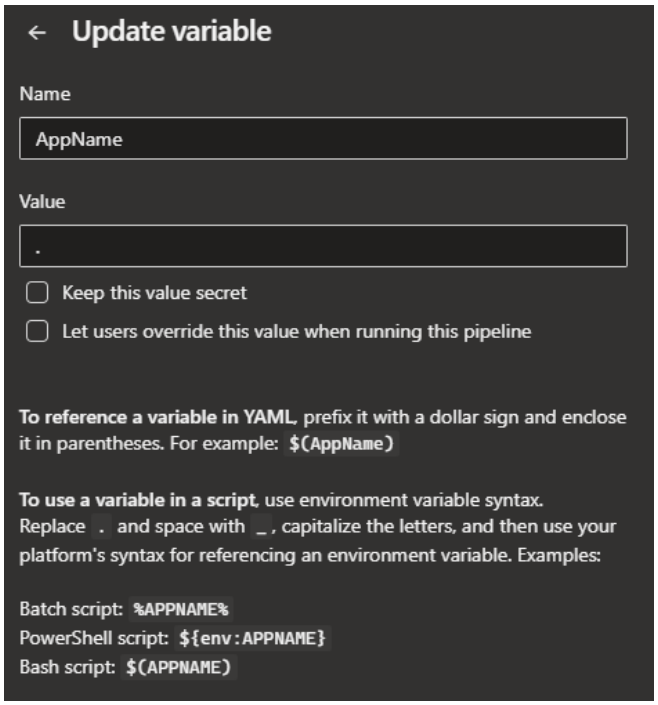
Las configuraciones para tener en cuenta son:

- **Federated {federated_oidc_provider_arn}**, es el ARN del OIDC del EKS, el valor se encuentra en el archivo vars del proyecto de capa media de CrossChannel.
  Tiene una forma similar a *arn:aws:iam::123456789:oidc-provider/oidc.eks.us-east-1.amazonaws.com/id/ABCDEFG123456*
- **{oidc_provider_url}**, es el OIDC del EKS, su valor tambien se encuentra en el archivo vars para los diferentes ambientes.
  Tiene una forma similar a *oidc.eks.us-east-1.amazonaws.com/id/ABCDEFG123456*
- **system:serviceaccount:${namespace}:push-info-mgmt-service-account:** En esta sección es importante que el nombre final, sea exactamente el mismo nombre del ServiceAccount creado en los manifiestos de k8s.
- **Recursos a los que se aplican las políticas:** En el caso de las políticas de cifrado, se aplican al recurso de KMS según el ambiente en que se estén desplegando. En CrossChannel se maneja un KMS único para el ambiente DEV y KMS separados por componentes para los ambientes QA y PROD.
  En el caso de los recursos de DynamoDb, el ejemplo muestra una manera en que las políticas se apliquen a las tablas y a sus índices.

Configuración de CI/CD

Modificaciones al CI en Azure

Para el CI en la plataforma de Azure únicamente es necesario cambiar la variable de AppName, agregando un punto.



Modificaciones al CD

Ambiente DEV

Para el despliegue es necesario tener dos stages, uno que despliega la imagen de la aplicación en el ECR y otro que despliega la aplicación en el EKS.

Para establecer estos pasos se tomó de base la guía en la WIKI.

Es necesario comprender algunos conceptos:

- **Docker registry service connection:** (Conexión de servicio de registro de Docker) es un puente seguro en herramientas de CI/CD como Azure DevOps que almacena credenciales de acceso. Permite que tus tareas automatizadas descarguen o suban imágenes de contenedores sin exponer claves secretas en el código.
- **Kubernetes service connection:** (Conexión de servicio de Kubernetes) es una configuración de credenciales y permisos que permite a plataformas de integración continua, como Azure DevOps, conectarse de forma segura a un

clúster de Kubernetes para desplegar y administrar aplicaciones.

Para CrossChannel existen un Docker Registry Service Connection por ambiente:

- aws-ba-001-CrossChannel-ecr-dev
- aws-ba-001-CrossChannel-ecr-qa
- Pendiente el de producción

Y 2 Kubernetes Service Connection por ambiente:

- eks-fullstack-crosschannel-dev
- eks-operacion-crosschannel-dev

- eks-fullstack-crosschannel-qa
- eks-operacion-crosschannel-qa

- Pendientes los de producción

Variables

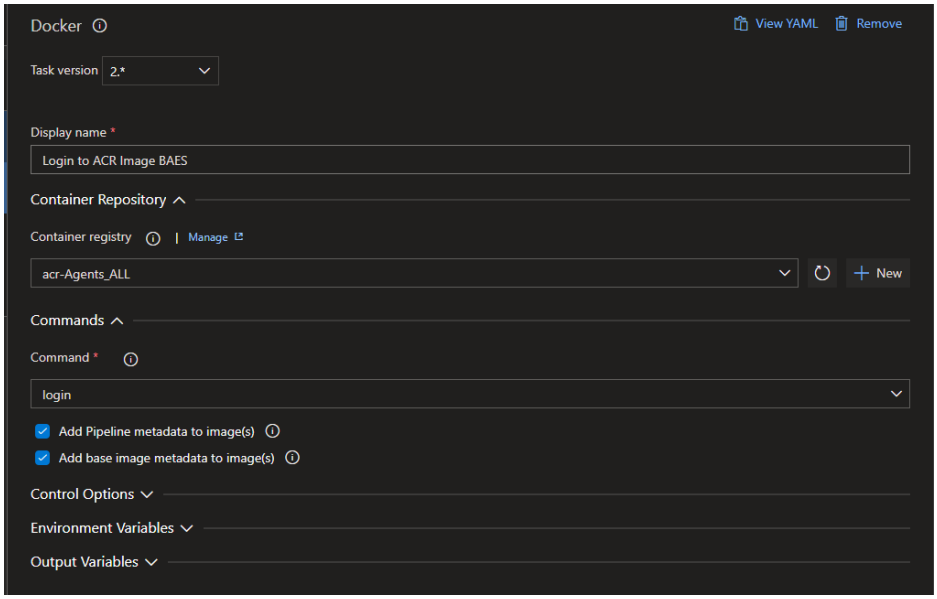
En la sección de variables necesitaremos crear variables que se utilizaran en las tareas de los CD. Estas son:

- **name:** Nombre del API que debe coincidir con el utilizado en las variables de los manifiestos de k8s.
- **repository:** Es la ruta del ECR según el ambiente
    - Para CrossChannel DEV: 730335499576.dkr.ecr.us-east-1.amazonaws.com/ecr-dev-mysismstraza-mcjzkr4ltij3
    - Para CrossChannel QA: 180250392861.dkr.ecr.us-east-1.amazonaws.com/ecr-qa-mysismstraza-1sq0ofpjajvx
    - Pendiente el de producción
- **repository_ecr:** Es el nombre del repositorio según el ambiente
    - Para CrossChannel DEV: ecr-dev-mysismstraza-mcjzkr4ltij3
    - Para CrossChannel QA: ecr-qa-mysismstraza-1sq0ofpjajvx
    - Pendiente el de producción
- **namespace:** Espacio de nombres del cluster EKS por ambiente
    - devmtls-crosschannel, para DEV
    - qamtls-crosschannel, para QA
    - Pendiente el de producción

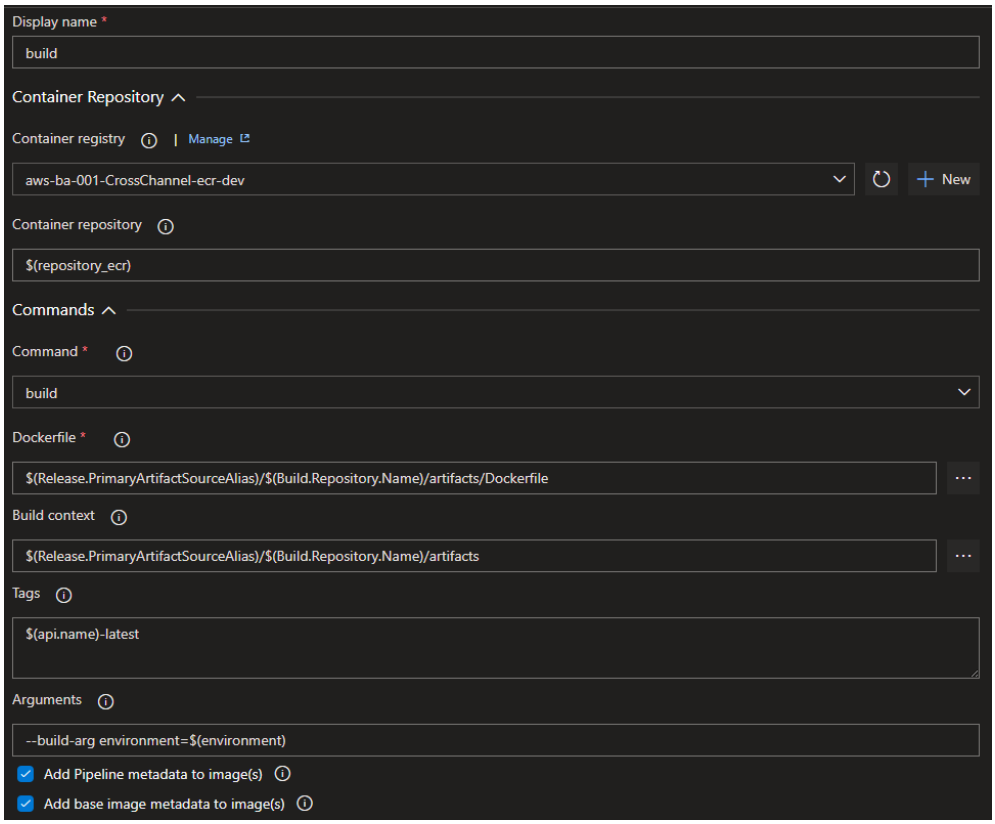
Stage de subida de imagen

El stage consiste en 3 tareas:

Login to ACR Image BAES

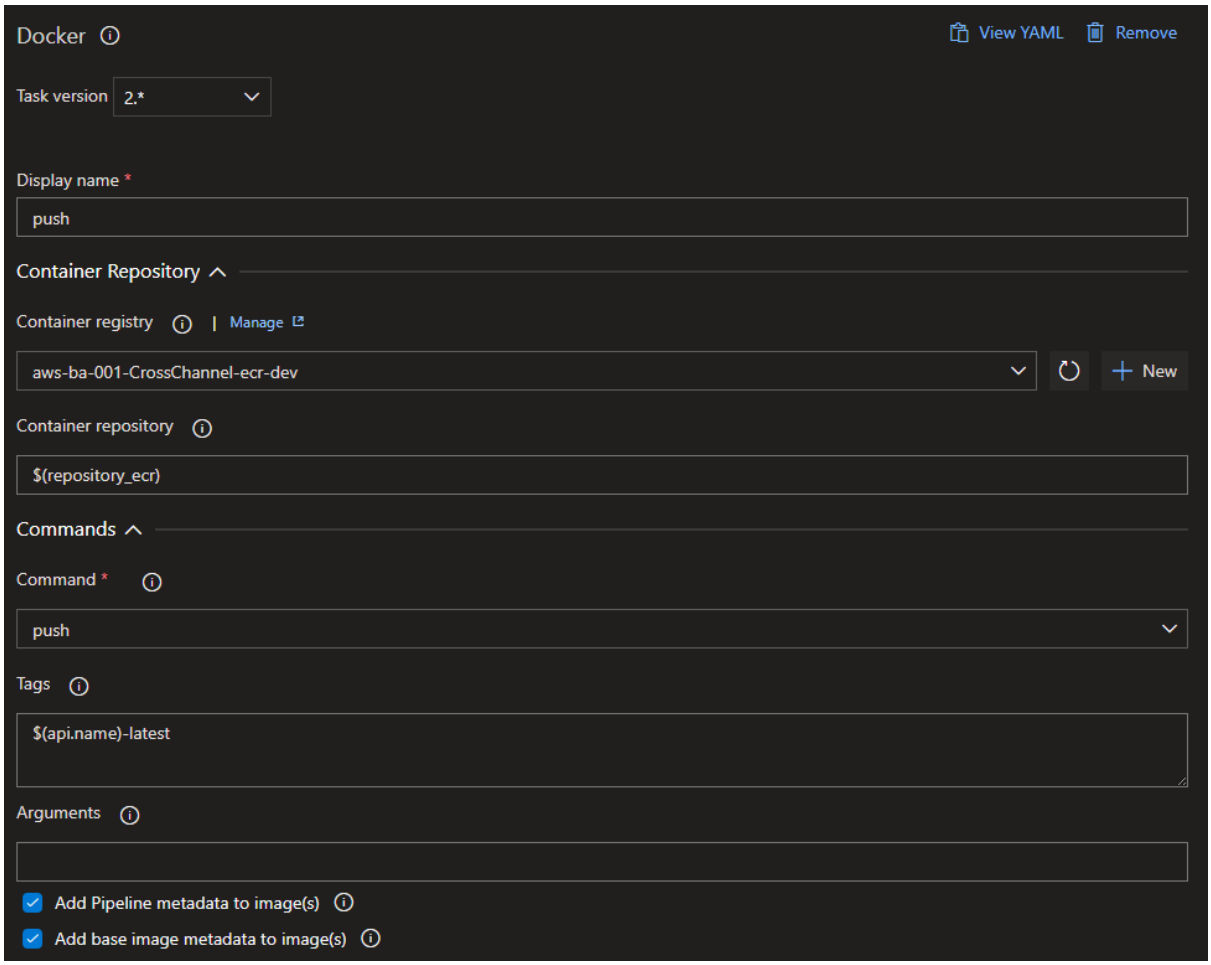


Build



El campo de **Tags** establece el nombre de la imagen, es este nombre el que se respeta al declarar la imagen en el manifiesto de k8s.

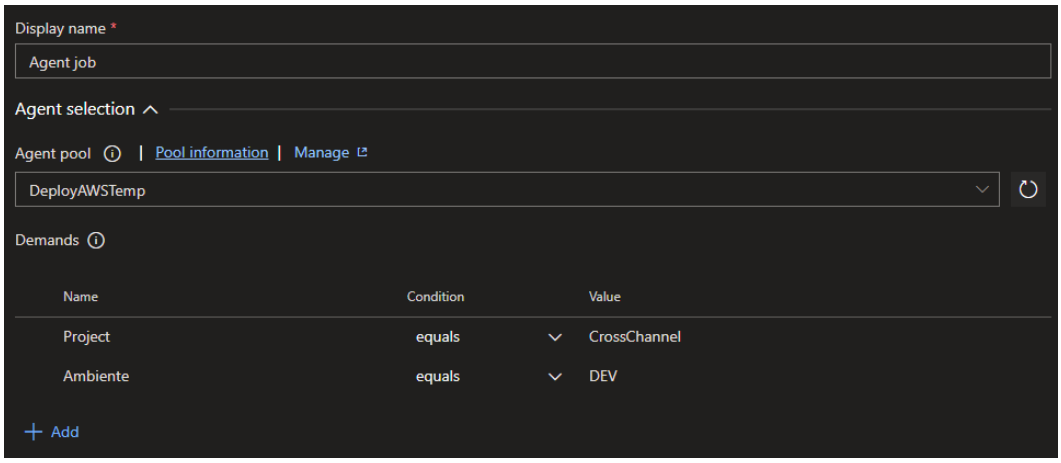
Push



El campo de **Tags** establece el nombre de la imagen, es este nombre el que se respeta al declarar la imagen en el manifiesto de k8s.

Stage de despliegue de API

Es importante configurar la tarea de *Agent job* de la siguiente manera para que el pipeline utilice el agente correcto.

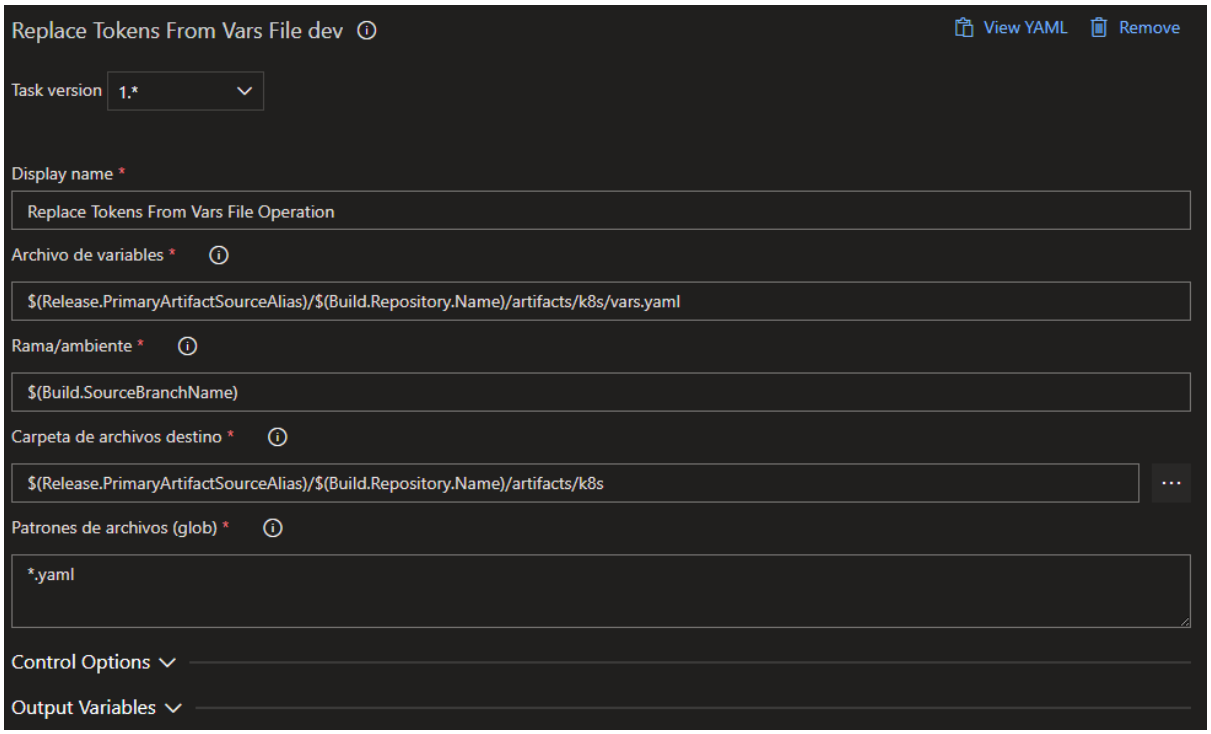


(En el caso del pipeline de QA, el valor de la variable de ambiente es **QA**)

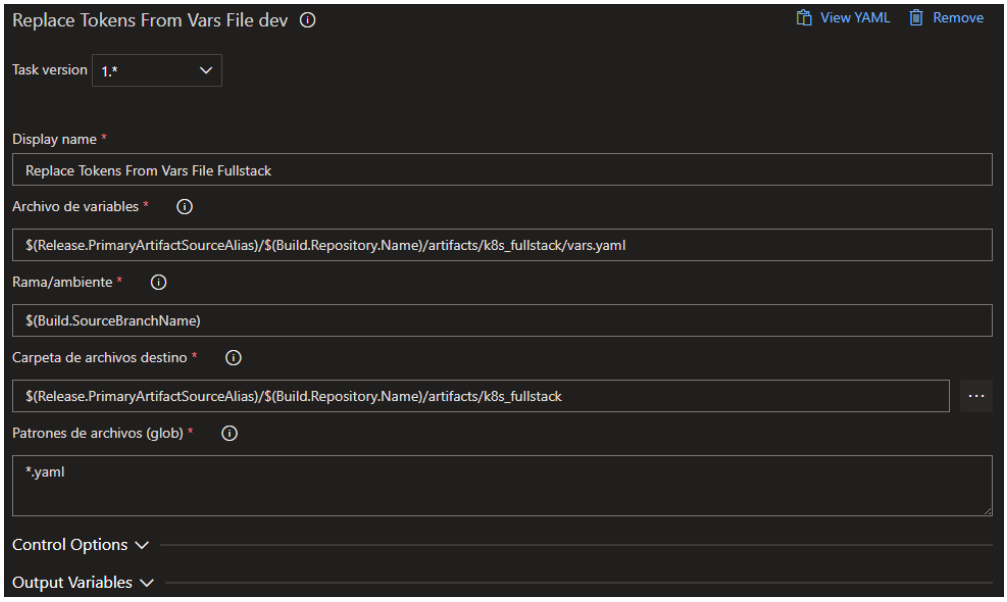
Es importante que en todas las tareas de kubectl, dentro de las opciones avanzadas se active el check de *Check for latest version*.

El stage consiste en 9 tareas:

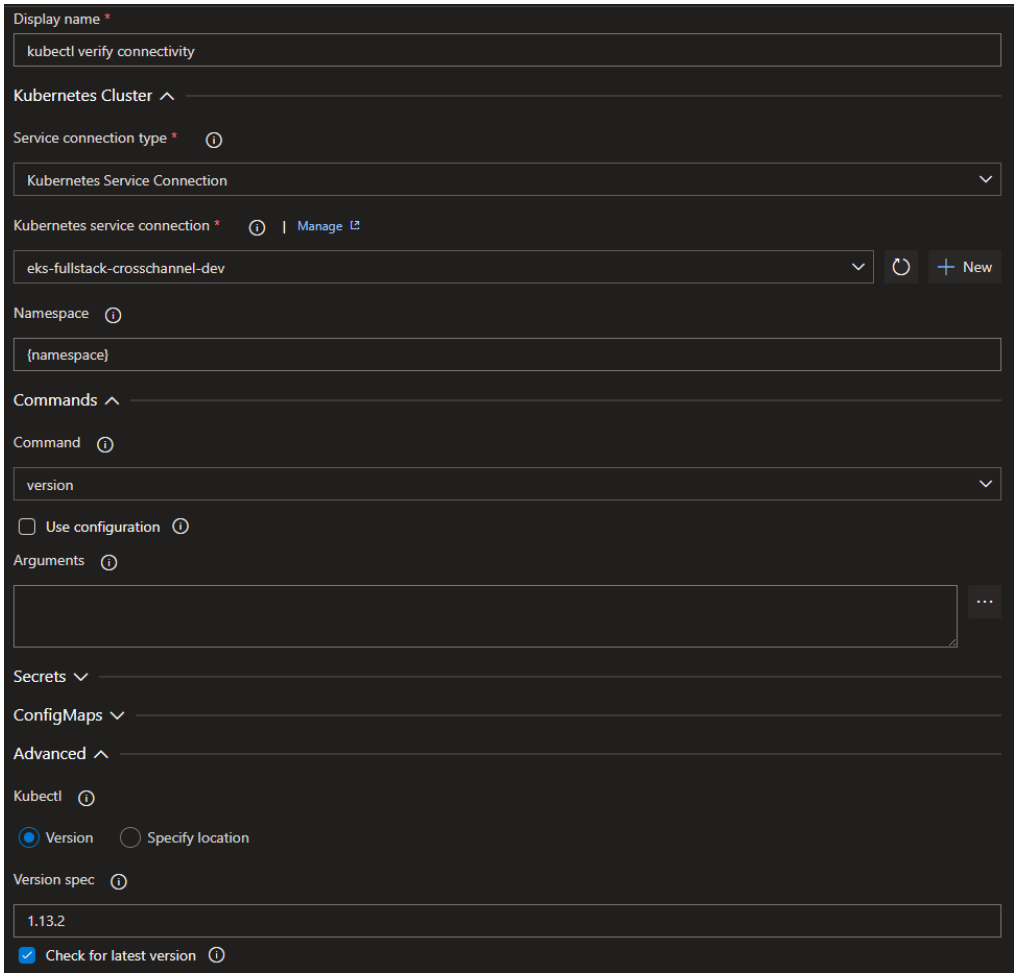
- **Replace Tokens From Vars File Operation:** Trabaja en la carpeta k8s_operacion definida y asigna las variables correspondientes a estos archivos que trabajan con el Service Conection de *Operation*.



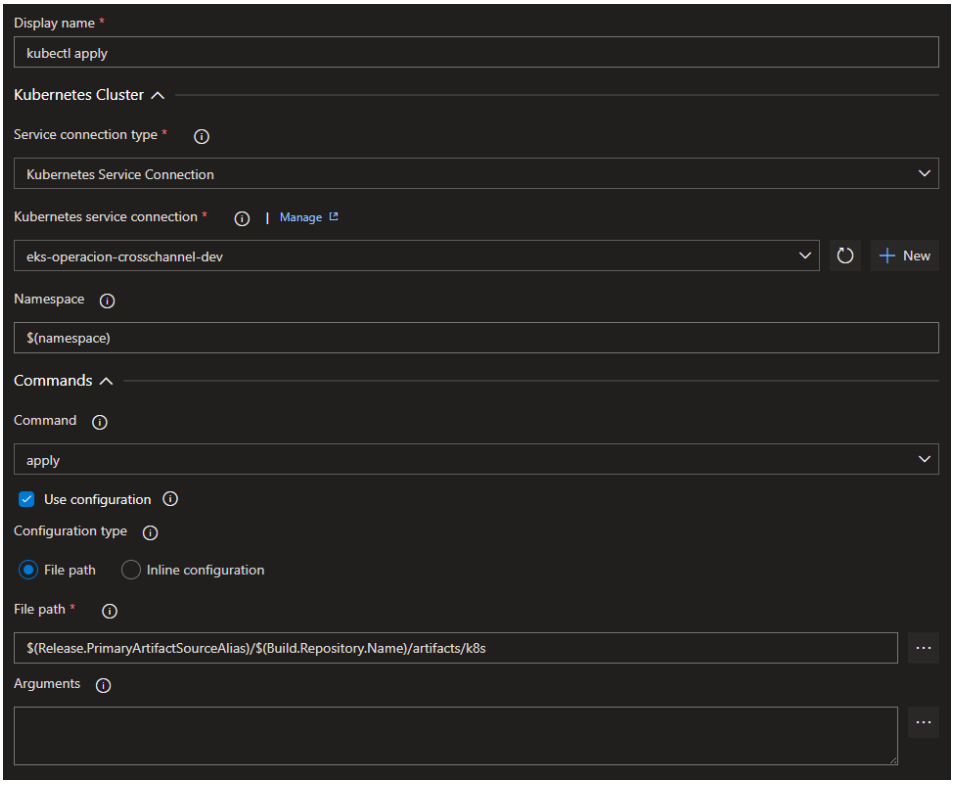
- **Replace Tokens From Vars File Fullstack:** Trabaja en la carpeta k8s_fullstack definida y asigna las variables correspondientes a estos archivos que trabajan con el Service Conection de *Fullstack*.



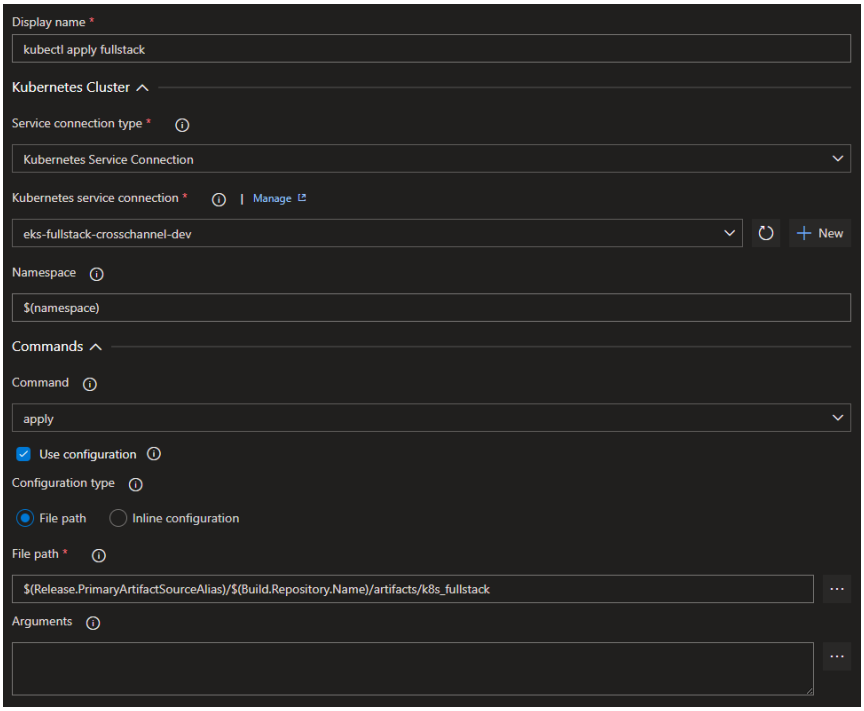
- **kubectl verify connectivity:** Ejecuta un comando de consulta de versión para confirmar que se tiene conexión hacia el EKS.



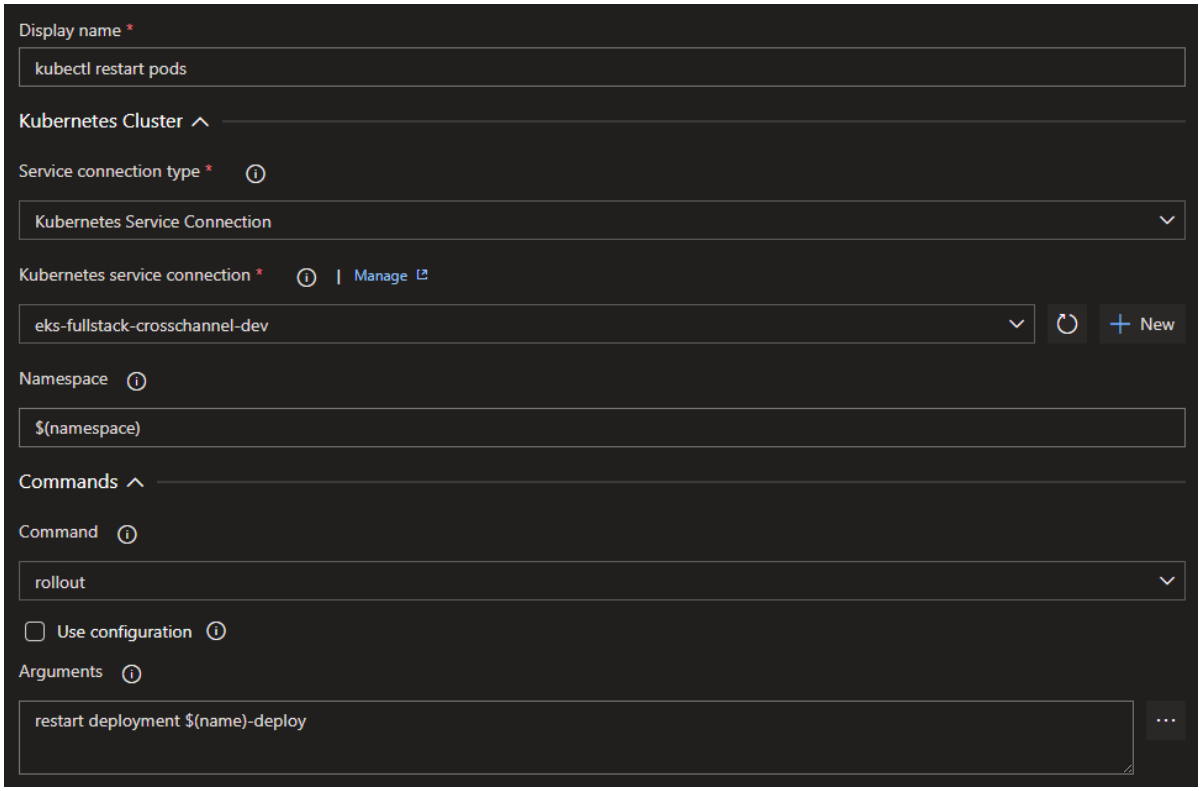
- **kubectl apply:** Despliega todos los manifiestos de la carpeta k8s_operacion que necesitan del Service Connection de *Operation*.



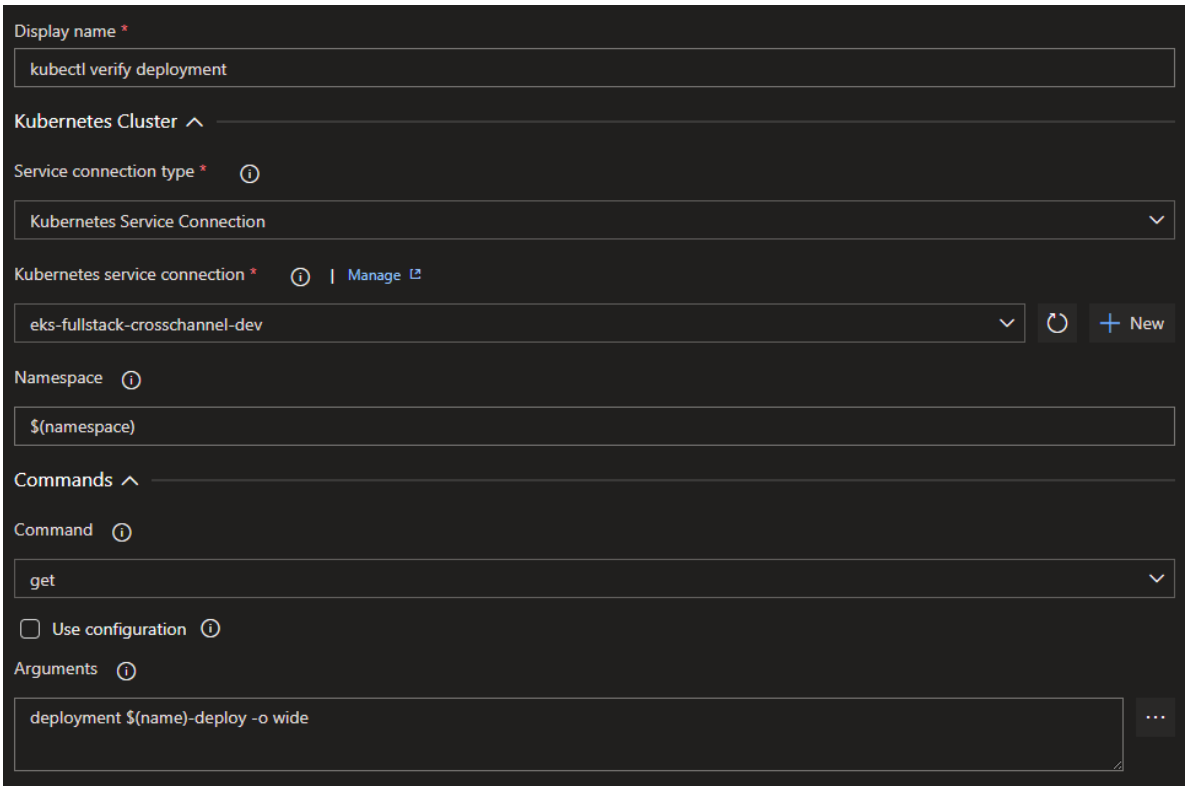
- **kubectl apply fullstack:** Despliega todos los manifiestos de la carpeta k8s_fullstack que necesitan del Service Connection de *Fullstack*. Esta separación entre operation y fullstack existe ya que los manifiestos de esta carpeta únicamente se pueden desplegar con el Service Connection mencionado.



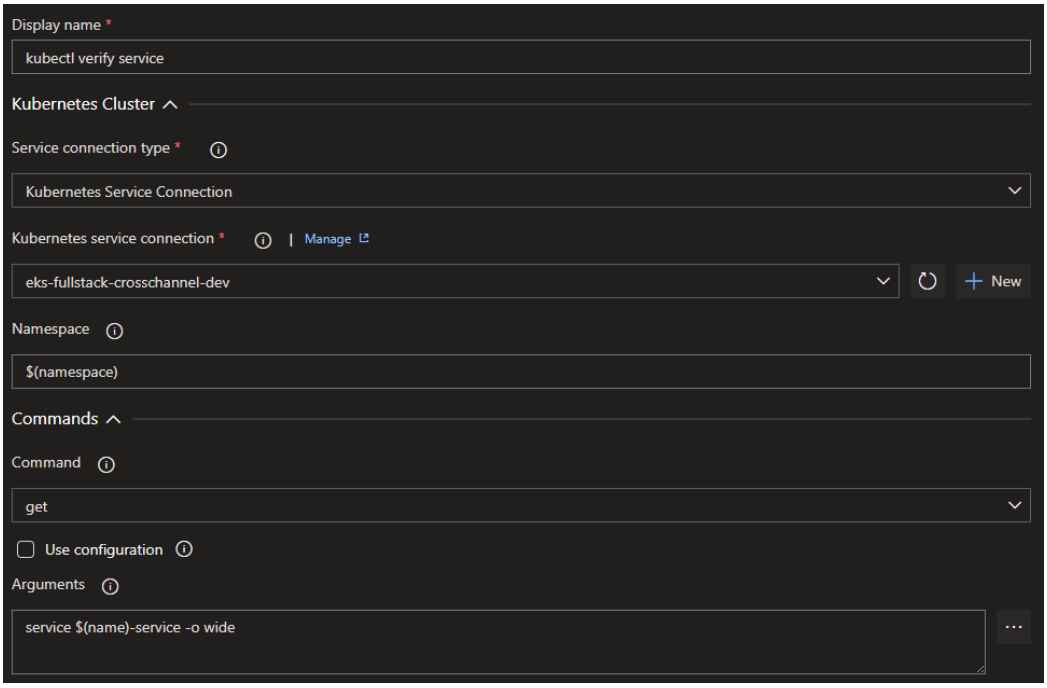
- **kubectl restart pods:** Reinicia los pods instalados.



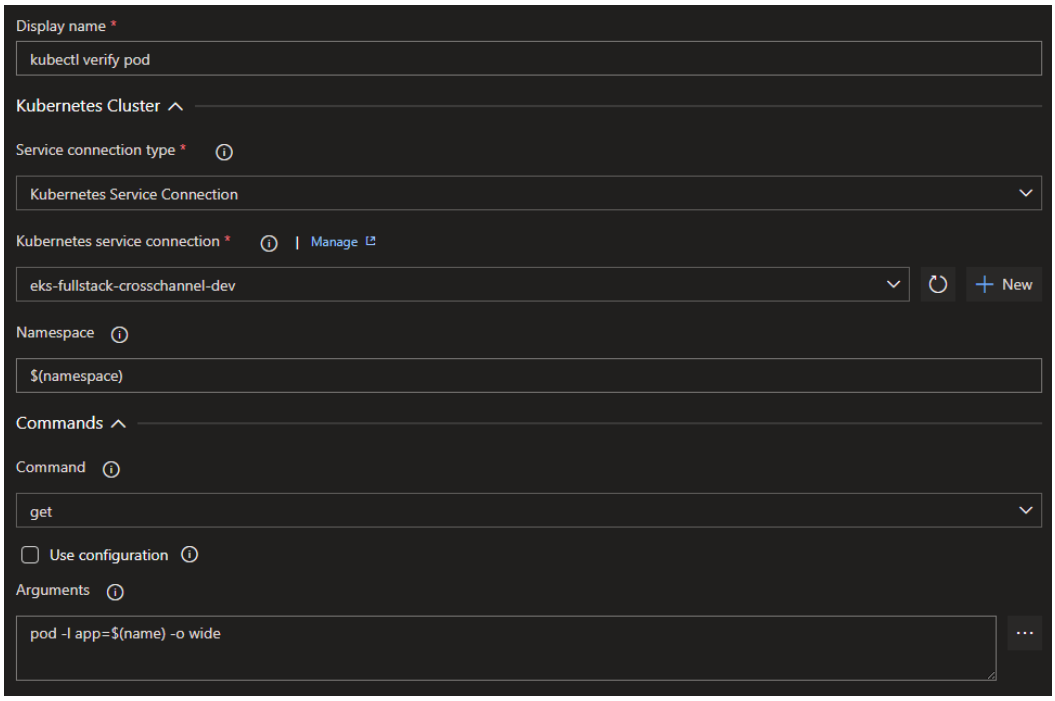
- **kubectl verify deployment:** Confirma que el manifiesto de *deployment* se haya desplegado.



- **kubectl verify service:** Confirma que el manifiesto de Service se haya desplegado.



- **kubectl verify pod:** Muestra los pods instalados y su estado.



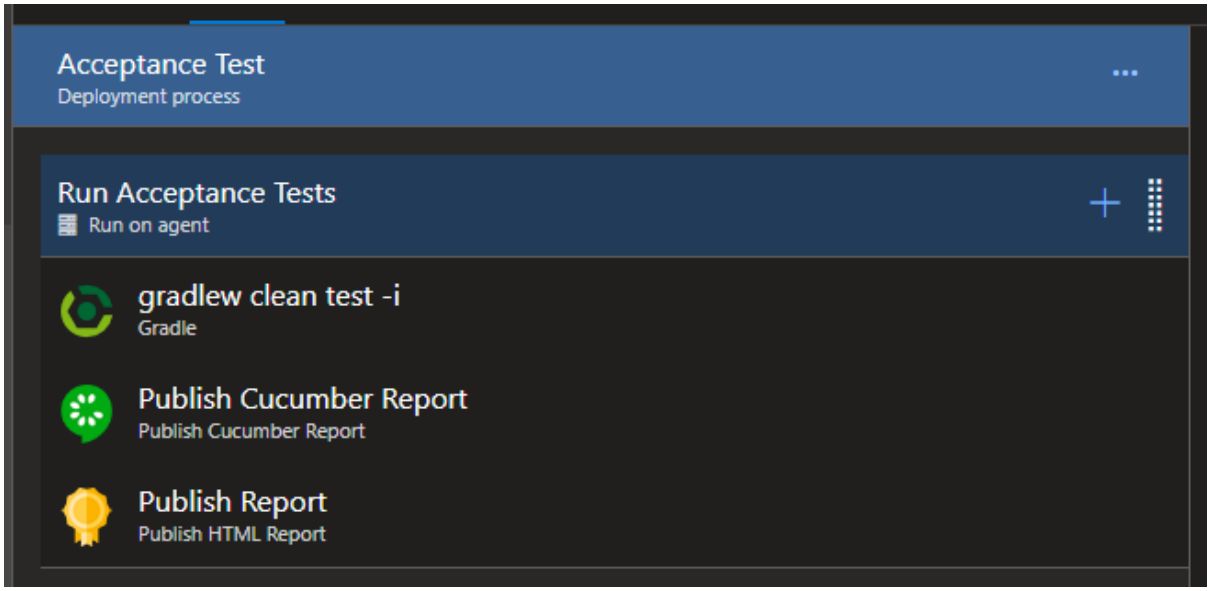
Si el despliegue fue correcto, el resultado de la ejecución debería mostrar los pods listos y con estado de Running para los pods.

```
[command]/home/devops/agentesAzure/_work/_tool/kubectl/1.36.3/x64/kubectl g
NAME                                   READY   STATUS    RESTARTS   AGE
push-info-mgmt-deploy-5fdcf5b7c6-pdww1   2/2     Running   0          59s
```

Ambiente QA

Para el despliegue son necesarias los stages descritos en el ambiente DEV, con la diferencia que se usaran los SVC propios de QA tanto para el stage de subida de imagen al ECR, como para el despliegue de los pods en el EKS.

Adicional, por lineamientos de devops debe existir el stage de **Acceptance Tests** descrito en este artículo de la WIKI. A la fecha de creación de este stage, no existe información mayor y es un stage de ejecución manual que solo algunos equipos están ejecutando.




TURNO DE...
PREGUNTAS (fáciles)
COMENTARIOS (bonitos)
CRÍTICAS (constructivas)
APORTACIONES (monetarias)